Technical Reference

Database Relations

SecureAIExam — every foreign-key relationship in the schema, what it means, and what happens on delete.

1. How the Schema Fits Together

Everything traces back to one root: an account in `auth.users`, extended by a `profiles` row that carries its role. From there, the schema branches into exam definition, the question bank, papers and their encrypted secrets, physical copies, and the custody/oversight trail that watches over all of it.

```
auth.users
\-- profiles                    (role: admin / teacher / paper_setter / invigilator)
    +-- setter_domains          (which exam categories a Setter may work in)
    +-- questions               (teacher_id)
    \-- exams                   (created_by)
        +-- exam_slots          (the day's timing windows)
        +-- exam_centers        (slot_id, auth_user_id -> invigilator login)
        |    \-- paper_copies   (center_id, once allocated)
        +-- teacher_subjects    (exam_id + teacher_id + subject)
        +-- questions           (exam_id -- which exam a question belongs to)
        \-- papers               (exam_id, setter_id)
            +-- paper_questions (paper_id + question_id -- junction)
            +-- paper_secrets   (paper_id -- 1:1, ciphertext + key shares)
            \-- paper_copies    (paper_id -- one row per physical copy)
                +-- custody_events   (copy_id -- hash-chained history)
                \-- qr_detections    (copy_id -- per-question watermark sightings)

alerts          -> papers (optional), paper_copies (optional)
qr_detections   -> exams, papers, paper_copies (all optional -- best-effort context)
audit_log       -> profiles, by actor_id  (soft reference only -- see note, section 4)
```

Reading the tree: a line further right depends on the line above it. Deleting an `exam` cascades down through its slots, centers, and papers; deleting a `paper` cascades down through its questions-link, its secret, and its printed copies; deleting a `paper_copy` cascades down through its own custody history.

2. Every Relationship, in Full

Read each row as: *the column on the left points to the row it names on the right.* `many→1` means many rows on the left can point to the same one row on the right — the normal case for a foreign key.

From	Column	References	Type	On Delete	What it Means
`profiles`	`id`	`auth.users.id`	`1:1`	`cascade`	A profile *is* the login account, extended with a role and approval status.
`questions`	`teacher_id`	`profiles.id`	`many→1`	—	Every question belongs to the teacher who submitted it.
`questions`	`exam_id`	`exams.id`	`many→1`	—	Which exam's bank a question was submitted into.
`exams`	`created_by`	`profiles.id`	`many→1`	—	The Admin who scheduled the exam.
`exams`	`paper_id`	`papers.id`	`many→1`	`set null`	Legacy convenience pointer to one paper; the authoritative link is `papers.exam_id` below.
`exam_slots`	`exam_id`	`exams.id`	`many→1`	`cascade`	A day's 2–3 timing windows, each tied to one exam.
`exam_centers`	`exam_id`	`exams.id`	`many→1`	`cascade`	Which exam a physical center is running.
`exam_centers`	`slot_id`	`exam_slots.id`	`many→1`	—	Which timing window that center runs.
`exam_centers`	`auth_user_id`	`auth.users.id`	`1:1`	—	The provisioned invigilator login for that center.
`exam_centers`	`created_by`	`profiles.id`	`many→1`	—	The Admin who provisioned the center.
`teacher_subjects`	`exam_id`	`exams.id`	`junction`	`cascade`	Composite key `(exam_id, teacher_id, subject)` — which subjects a teacher may write for, per exam.
`teacher_subjects`	`teacher_id`	`profiles.id`	`junction`	`cascade`	The other half of the same composite key.
`teacher_subjects`	`assigned_by`	`profiles.id`	`many→1`	—	The Admin who made the assignment.
`setter_domains`	`setter_id`	`profiles.id`	`many→1`	`cascade`	Which exam categories a Paper Setter is authorized for — account-wide, not per exam.

setter_domains	assigned_by	profiles.id	many→1	—	The Admin who granted that domain (null for the two chosen at sign-up).
papers	exam_id	exams.id	many→1	—	Which exam this paper set belongs to.
papers	setter_id	profiles.id	many→1	—	The Paper Setter composing and sealing this set.
paper_questions	paper_id	papers.id	junction	cascade	Composite key (paper_id, question_id) — the many-to-many between papers and questions.
paper_questions	question_id	questions.id	junction	—	The other half of the same composite key.
paper_secrets	paper_id	papers.id	1:1	cascade	paper_id is itself the primary key — exactly one secret row per paper. No client-facing RLS policy exists on this table at all.
paper_copies	paper_id	papers.id	many→1	cascade	Every physical printed copy belongs to exactly one sealed paper.
paper_copies	center_id	exam_centers.	many→1	—	Which center a copy was allocated to (null until allocation).
paper_copies	scanned_by	profiles.id	many→1	—	The invigilator who scanned this copy in.
custody_events	copy_id	paper_copies.	many→1	cascade	Every checkpoint in one copy's hash-chained history.
custody_events	actor_id	profiles.id	many→1	—	Who recorded that checkpoint.
qr_detections	copy_id	paper_copies.	many→1	set null	Which copy a watermark sighting resolved to.
qr_detections	paper_id	papers.id	many→1	set null	Denormalized for fast lookups without a join.
qr_detections	exam_id	exams.id	many→1	set null	Denormalized, same reason.
qr_detections	reported_by	profiles.id	many→1	—	Who or what device reported the sighting.
alerts	paper_id	papers.id	many→1	set null	Which paper an alert concerns, if any.
alerts	copy_id	paper_copies.	many→1	set null	Which physical copy an alert concerns, if any.

3. Grouped by Relationship Shape

One-to-one

Relationship	Why it's 1:1
`profiles.id` ↔ `auth.users.id`	A profile doesn't exist independently of its login account — it's the same identity, extended.
`paper_secrets.paper_id` ↔ `papers.id`	`paper_id` is the primary key of `paper_secrets` itself, so there can never be more than one secret row per paper.
`exam_centers.auth_user_id` ↔ `auth.users.id`	Each center gets exactly one provisioned invigilator login.

One-to-many (the majority of the schema)

One `exam` has many `exam_slots`, many `exam_centers`, and many `papers` (one per set). One `paper` has many `paper_copies` (one per printed copy). One `paper_copy` has many `custody_events` (one per checkpoint) and many `qr_detections` (one per watermark sighting). One `profile` has many `questions`, many `papers` (if a Setter), and many `setter_domains`.

Many-to-many (via a junction table)

Junction Table	Connects	Composite Primary Key
`paper_questions`	`papers` ↔ `questions`	`(paper_id, question_id)`
`teacher_subjects`	`exams` ↔ `profiles` (teachers)	`(exam_id, teacher_id, subject)`
`setter_domains`	`profiles` (setters) ↔ exam categories	`(setter_id, subject)`

4. Special Cases Worth Knowing

Cascading deletes follow the ownership chain

Delete an `exam` and its slots, centers, and papers go with it. Delete a `paper` and its question links, its encrypted secret, and its printed copies go with it. Delete a `paper_copy` and its own custody history goes with it. This mirrors real-world ownership: none of those child records mean anything without their parent.

"Set null" instead of cascade, where the record should survive

`alerts` and `qr_detections` point to papers and copies with `on delete set null`, not cascade. An alert or a forensic sighting is a historical fact — it should keep existing even if the paper it was about is later removed, just with that link cleared rather than the whole row disappearing.

`audit_log.actor_id` is a soft reference, not an enforced foreign key

Unlike every other actor column in the schema, `audit_log` does not declare a real foreign-key constraint on `actor_id` — it also stores `actor_name` directly, as a snapshot. This is deliberate: the audit log must remain a permanent, readable record forever, even if the acting account is later deleted. A hard foreign key would either block that deletion or silently erase the actor's name from history — neither is acceptable for an audit trail.

`paper_copies.center_id` is added after the table, as a named constraint

This one is added via a separate `alter table … add constraint` statement rather than inline, purely because of the order tables are created in the schema file (`exam_centers` is defined after `paper_copies`). Functionally it behaves exactly like any other foreign key.

> **The one table with no relations pointing outward at all:** `paper_secrets` never appears on the "From" side of any relationship other than its own primary key. It only ever gets pointed *at* — nothing about it is designed to be joined outward, which matches its job: hold ciphertext and wrapped key shares, and nothing else.